



Reference Data

RV32i BASE INTEGER INSTRUCTIONS, in alphabetical order				
MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
add	R	ADD	$R[d] = R[rs1] + R[rs2]$	
addi	I	ADD Immediate	$R[d] = R[rs1] + imm$	
and	R	AND	$R[d] = R[rs1] \& R[rs2]$	
andi	I	AND Immediate	$R[d] = R[rs1] \& imm$	
auipc	U	Add Upper Immediate to PC	$R[d] = PC + \{imm, 12'b0\}$	
beq	SB	Branch Equal	$if(R[rs1] == R[rs2])$ $PC = PC + \{imm, 1b'0\}$	
bge	SB	Branch Greater than or Equal	$if(R[rs1] \geq R[rs2])$ $PC = PC + \{imm, 1b'0\}$	
bgeu	SB	Branch $\geq$ Unsigned	$if(R[rs1] \geq R[rs2])$ $PC = PC + \{imm, 1b'0\}$	2)
blt	SB	Branch Less Than	$if(R[rs1] < R[rs2])$ $PC = PC + \{imm, 1b'0\}$	
bltu	SB	Branch Less Than Unsigned	$if(R[rs1] < R[rs2])$ $PC = PC + \{imm, 1b'0\}$	2)
bne	SB	Branch Not Equal	$if(R[rs1] \neq R[rs2])$ $PC = PC + \{imm, 1b'0\}$	
csrrc	I	Cont./Stat.RegRead&Clear	$R[d] = CSR; CSR = CSR \& \sim R[rs1]$	
csrrci	I	Cont./Stat.RegRead&Clear Imm	$R[d] = CSR; CSR = CSR \& \sim imm$	
csrrs	I	Cont./Stat.RegRead&Set	$R[d] = CSR; CSR = CSR R[rs1]$	
csrrsi	I	Cont./Stat.RegRead&Set Imm	$R[d] = CSR; CSR = CSR imm$	
csrrw	I	Cont./Stat.RegRead&Write	$R[d] = CSR; CSR = R[rs1]$	
csrrwi	I	Cont./Stat.RegRead&Write Imm	$R[d] = CSR; CSR = imm$	
ebreak	I	Environment BREAK	Transfer control to debugger	
ecall	I	Environment CALL	Transfer control to operating system	
fence	I	Synch thread	Synchronizes threads	
fence.i	I	Synch Instr. & Data	Synchronizes writes to instruction stream	
jal	UJ	Jump & Link	$R[d] = PC+4; PC = PC + \{imm, 1b'0\}$	
jalr	I	Jump & Link Register	$R[d] = PC+4; PC = R[rs1] + imm$	
lb	I	Load Byte	$R[d] = \{24'b0, M[R[rs1] + imm][7:0]\}$	3)
lbu	I	Load Byte Unsigned	$R[d] = \{24'b0, M[R[rs1] + imm][7:0]\}$	4)
lh	I	Load Halfword	$R[d] = \{16'b0, M[R[rs1] + imm][15:0]\}$	
lhu	I	Load Halfword Unsigned	$R[d] = \{16'b0, M[R[rs1] + imm][15:0]\}$	4)
lui	U	Load Upper Immediate	$R[d] = \{imm, 12'b0\}$	
lw	I	Load Word	$R[d] = \{M[R[rs1] + imm][31:0]\}$	
or	R	OR	$R[d] = R[rs1] R[rs2]$	
ori	I	OR Immediate	$R[d] = R[rs1] imm$	4)
sb	S	Store Byte	$M[R[rs1] + imm][7:0] = R[rs2][7:0]$	
sh	S	Store Halfword	$M[R[rs1] + imm][15:0] = R[rs2][15:0]$	
sll	R	Shift Left	$R[d] = R[rs1] << R[rs2]$	
slli	I	Shift Left Immediate	$R[d] = R[rs1] << imm$	
slt	R	Set Less Than	$R[d] = (R[rs1] < R[rs2]) ? 1 : 0$	
slti	I	Set Less Than Immediate	$R[d] = (R[rs1] < imm) ? 1 : 0$	
sltui	I	Set < Immediate Unsigned	$R[d] = (R[rs1] < imm) ? 1 : 0$	
sltu	R	Set Less Than Unsigned	$R[d] = (R[rs1] < R[rs2]) ? 1 : 0$	
sra	R	Shift Right Arithmetic	$R[d] = R[rs1] >> R[rs2]$	
srai	I	Shift Right Arith Imm	$R[d] = R[rs1] >> imm$	2)
srl	R	Shift Right (Word)	$R[d] = R[rs1] >> R[rs2]$	
srli	I	Shift Right Immediate	$R[d] = R[rs1] >> imm$	5)
sub, subw	R	SUBtract (Word)	$R[d] = R[rs1] - R[rs2]$	
sw	S	Store Word	$M[R[rs1] + imm][31:0] = R[rs2][31:0]$	
xor	R	XOR	$R[d] = R[rs1] \wedge R[rs2]$	
xori	I	XOR Immediate	$R[d] = R[rs1] \wedge imm$	

- Notes: 1) Operation assumes unsigned integers (instead of 2's complement)
2) The least significant bit of the branch address in jalr is set to 0
3) (signed) Load instructions extend the sign bit of data to fill the 32-bit register
4) Replicates the sign bit to fill in the leftmost bits of the result during right shift
5) Multiply with one operand signed and one unsigned
6) The Single version does a single-precision operation using the rightmost 32 bits of a 64-bit F register
7) Classifies ways a 10-bit mask to show which properties are true (e.g., -inf, -0, +0, +inf, denorm, ...)
8) Atomic memory operation; nothing else can interpose itself between the read and the write of the memory location
The immediate field is sign-extended in RISC-V

①

ARITHMETIC CORE INSTRUCTION SET

RV64M Multiply Extension

MNEMONIC	FMT	NAME	DESCRIPTION (in Verilog)	NOTE
mul	R	MULtiply	$R[d] = (R[rs1] * R[rs2]) \> 63:0$	
mulh	R	MULtiply High	$R[d] = (R[rs1] * R[rs2])[127:64]$	
mulhsu	R	MULtiply High Unsigned	$R[d] = (R[rs1] * R[rs2])[127:64]$	2)
mulhu	R	MULtiply upper Half Unsigned	$R[d] = (R[rs1] * R[rs2])[127:64]$	6)
div	R	DIVide	$R[d] = (R[rs1] / R[rs2])$	
divu	R	DIVide Unsigned	$R[d] = (R[rs1] / R[rs2])$	2)
rem	R	REMAinder	$R[d] = (R[rs1] \% R[rs2])$	
remu	R	REMAinder Unsigned	$R[d] = (R[rs1] \% R[rs2])$	2)

RV64F and RV64D Floating-Point Extensions

fld, fldw	I	Load (Word)	$F[d] = M[R[rs1] + imm]$	
fsw, fsw	S	Store (Word)	$M[R[rs1] + imm] = F[rd]$	
fadd.s, fadd.d	R	ADD	$F[d] = F[rs1] + F[rs2]$	7)
fsub.s, fsub.d	R	SUBtract	$F[d] = F[rs1] - F[rs2]$	7)
fmul.s, fmul.d	R	MULtiply	$F[d] = F[rs1] * F[rs2]$	7)
fdiv.s, fdiv.d	R	DIVide	$F[d] = F[rs1] / F[rs2]$	7)
fsqrt.s, fsqrt.d	R	SQare Root	$F[d] = \sqrt{F[rs1]}$	7)
fmad.s, fmad.d	R	MULtiply-ADD	$F[d] = F[rs1] * F[rs2] + F[rs3]$	7)
fmsub.s, fmsub.d	R	MULtiply-SUBtract	$F[d] = F[rs1] * F[rs2] - F[rs3]$	7)
fminsub.s, fminsub.d	R	Negative MULtiply-ADD	$F[d] = -F[rs1] * F[rs2] - F[rs3]$	7)
fmaxadd.s, fmaxadd.d	R	Negative MULtiply-SUBtract	$F[d] = -F[rs1] * F[rs2] + F[rs3]$	7)
fsgn.s, fsgn.d	R	SIGN source	$F[d] = \{F[rs2][63:0], F[rs1][62:0]\}$	7)
fsgnj.s, fsgnj.d	R	Negative SIGN source	$F[d] = \{F[rs2][63:0], F[rs1][62:0]\}$	7)
fsgnjx.s, fsgnjx.d	R	Xor SIGN source	$F[d] = \{F[rs2][63:0] \oplus F[rs1][62:0], F[rs1][62:0]\}$	7)
fmin.s, fmin.d	R	MINimum	$F[d] = (F[rs1] < F[rs2]) ? F[rs1] : F[rs2]$	7)
fmax.s, fmax.d	R	MAXimum	$F[d] = (F[rs1] > F[rs2]) ? F[rs1] : F[rs2]$	7)
feq.s, feq.d	R	Compare Float Equal	$R[d] = (F[rs1] == F[rs2]) ? 1 : 0$	7)
flt.s, flt.d	R	Compare Float Less Than	$R[d] = (F[rs1] < F[rs2]) ? 1 : 0$	7)
flt.s.le, flt.d.le	R	Compare Float Less than or =	$R[d] = (F[rs1] <= F[rs2]) ? 1 : 0$	7)
fcvt.s.w, fcvt.d.w	R	Classify Type	$R[d] = \text{class}(F[rs1])$	7,8)
fmv.s.x, fmv.d.x	R	Move from Integer	$F[d] = R[rs1]$	7)
fmv.x.s, fmv.x.d	R	Move to Integer	$R[d] = F[rs1]$	7)
fcvt.s.d	R	Convert from SP to DP	$F[d] = \text{single}(F[rs1])$	
fcvt.d.s	R	Convert from DP to SP	$F[d] = \text{double}(F[rs1])$	
fcvt.s.w, fcvt.d.w	R	Convert from 32b Integer	$F[d] = \text{float}(R[rs1][31:0])$	7)
fcvt.s.l, fcvt.d.l	R	Convert from 64b Integer	$F[d] = \text{float}(R[rs1][63:0])$	7)
fcvt.s.wu, fcvt.d.wu	R	Convert from 32b Int Unsigned	$F[d] = \text{float}(R[rs1][31:0])$	2,7)
fcvt.s.lu, fcvt.d.lu	R	Convert from 64b Int Unsigned	$F[d] = \text{float}(R[rs1][63:0])$	2,7)
fcvt.w.s, fcvt.w.d	R	Convert to 32b Integer	$R[d][31:0] = \text{integer}(F[rs1])$	7)
fcvt.l.s, fcvt.l.d	R	Convert to 64b Integer	$R[d][63:0] = \text{integer}(F[rs1])$	7)
fcvt.wu.s, fcvt.wu.d	R	Convert to 32b Int Unsigned	$R[d][31:0] = \text{integer}(F[rs1])$	2,7)
fcvt.lu.s, fcvt.lu.d	R	Convert to 64b Int Unsigned	$R[d][63:0] = \text{integer}(F[rs1])$	2,7)

RV64A Atomic Extension

amoadd.w, amoadd.d	R	ADD	$R[d] = M[rs1]$	9)
amoand.w, amoand.d	R	AND	$M[rs1] = M[rs1] \& R[rs2]$	9)
amomax.w, amomax.d	R	MAXimum	$R[d] = M[rs1]$ $if(R[rs2] < M[rs1])$ $M[rs1] = R[rs2]$	2,9)
amomaxu.w, amomaxu.d	R	MAXimum Unsigned	$R[d] = M[rs1]$ $if(R[rs2] < M[rs1])$ $M[rs1] = R[rs2]$	2,9)
amomin.w, amomin.d	R	MINimum	$R[d] = M[rs1]$ $if(R[rs2] < M[rs1])$ $M[rs1] = R[rs2]$	2,9)
amominu.w, amominu.d	R	MINimum Unsigned	$R[d] = M[rs1]$ $if(R[rs2] < M[rs1])$ $M[rs1] = R[rs2]$	2,9)
amoor.w, amoor.d	R	OR	$R[d] = M[rs1]$	9)
amoswap.w, amoswap.d	R	SWAP	$R[d] = M[rs1]$ $R[rs2] = M[rs1]$	9)
amoxor.w, amoxor.d	R	XOR	$R[d] = M[rs1]$ $R[rs2] = M[rs1] \oplus R[rs2]$	9)
lr.w, lr.d	R	Load Reserved	$R[d] = M[rs1]$ reservation on $M[rs1]$ if reserved, $M[rs1] = R[rs2]$, $R[d] = 0$; else $R[d] = 1$	
sc.w, sc.d	R	Store Conditional		

CORE INSTRUCTION FORMATS

	31	27	26	25	24	20	19	15	14	12	11	7	6	0							
R	funct7					rs2			rs1			funct3		rd	Opcod						
I	imm[11:0]								rs1			funct3		rd	Opcod						
S	imm[11:5]					rs2			rs1			funct3		imm[4:0]	opcod						
SB	imm[12:0:5]					rs2			rs1			funct3		imm[4:11]	opcod						
U	imm[31:12]														rd	opcod					
UJ	imm[20:10:11:19:12]														rd	opcod					

PSEUDO INSTRUCTIONS

MNEMONIC	NAME	DESCRIPTION	USES
beqz	Branch = zero	if(R[rs1]==0) PC=PC+[imm,1b'0]	beq
bnez	Branch ≠ zero	if(R[rs1]!=0) PC=PC+[imm,1b'0]	bne
fabs.s, fabs.d	Absolute Value	F[rd] = (F[rs1]<0) ? -F[rs1] : F[rs1]	fsgnx
fmv.s, fmv.d	FP Move	F[rd] = F[rs1]	fsgnj
fneg.s, fneg.d	FP negate	F[rd] = -F[rs1]	fsgnjn
j	Jump	PC = (imm,1b'0)	jal
jr	Jump register	PC = R[rs1]	jalr
la	Load address	R[rd] = address	auipc
li	Load imm	R[rd] = imm	addi
mv	Move	R[rd] = R[rs1]	addi
neg	Negate	R[rd] = -R[rs1]	sub
nop	No operation	R[0] = R[0]	addi
not	Not	R[rd] = ~R[rs1]	xori
ret	Return	PC = R[1]	jalr
seqz	Set = zero	R[rd] = (R[rs1]==0) ? 1 : 0	sltiu
snez	Set ≠ zero	R[rd] = (R[rs1]!=0) ? 1 : 0	sltu

OPCODES IN NUMERICAL ORDER BY OPCODE

MNEMONIC	FMT	OPCODE	FUNCT3	FUNCT7 OR IMM	HEXADECIMAL
lb	I	0000011	000		03/0
lh	I	0000011	001		03/1
lw	I	0000011	010		03/2

lbu	I	0000011	100		03/4
lhu	I	0000011	101		03/5
fence	I	0001111	000		0F/0
fence.i	I	0001111	001		0F/1
addi	I	0010011	000		13/0
slli	I	0010011	001	0000000	13/1/00
slti	I	0010011	010		13/2
sltiu	I	0010011	011		13/3
xori	I	0010011	100		13/4
srl	I	0010011	101	0000000	13/5/00
srai	I	0010011	101	0100000	13/5/20
ori	I	0010011	110		13/6
andi	I	0010011	111		13/7
auipc	U	0010111			17

sb	S	0100011	000		23/0
sh	S	0100011	001		23/1
sw	S	0100011	010		23/2
add	R	0110011	000	0000000	33/0/00
sub	R	0110011	000	0100000	33/0/20
sll	R	0110011	001	0000000	33/1/00
slt	R	0110011	010	0000000	33/2/00
sltu	R	0110011	011	0000000	33/3/00
xor	R	0110011	100	0000000	33/4/00
srl	R	0110011	101	0000000	33/5/00
sra	R	0110011	101	0100000	33/5/20
or	R	0110011	110	0000000	33/6/00
and	R	0110011	111	0000000	33/7/00
lui	U	0110111			37

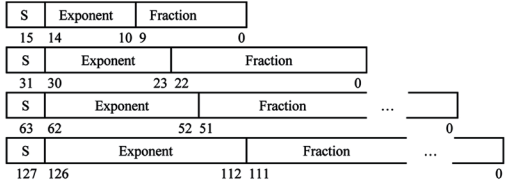
beq	SB	1100011	000		63/0
bne	SB	1100011	001		63/1
blt	SB	1100011	100		63/4
bge	SB	1100011	101		63/5
bltu	SB	1100011	110		63/6
bgeu	SB	1100011	111		63/7
jalr	I	1100111	000		67/0
jal	UJ	1101111			68
ecall	I	1110011	000	000000000000	73/0/000
ebreak	I	1110011	000	000000000001	73/0/001
CSRrw	I	1110011	001		73/1
CSRrs	I	1110011	010		73/2
CSRrc	I	1110011	011		73/3
CSRrwi	I	1110011	101		73/5
CSRrsi	I	1110011	110		73/6
CSRrci	I	1110011	111		73/7

REGISTER NAME, USE, CALLING CONVENTION

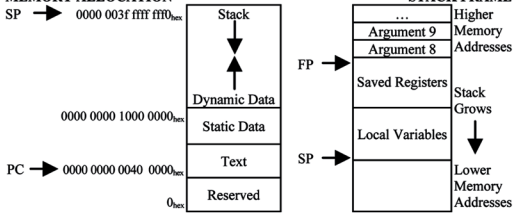
REGISTER	NAME	USE	SAVER
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/Return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP Temporaries	Caller
f8-f9	fs0-fs1	FP Saved registers	Callee
f10-f11	fa0-fa1	FP Function arguments/Return values	Caller
f12-f17	fa2-fa7	FP Function arguments	Caller
f18-f27	fs2-fs11	FP Saved registers	Callee
f28-f31	ft8-ft11	R[rd] = R[rs1] + R[rs2]	Caller

IEEE 754 FLOATING-POINT STANDARD

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$
where Half-Precision Bias = 15, Single-Precision Bias = 127,
Double-Precision Bias = 1023, Quad-Precision Bias = 16383
IEEE Half-, Single-, Double-, and Quad-Precision Formats:



MEMORY ALLOCATION



SIZE PREFIXES AND SYMBOLS

SIZE	PREFIX	SYMBOL	SIZE	PREFIX	SYMBOL
1000 ¹	Kilo-	K	2 ¹⁰	Kibi-	Ki
1000 ²	Mega-	M	2 ²⁰	Mebi-	Mi
1000 ³	Giga-	G	2 ³⁰	Gibi-	Gi
1000 ⁴	Tera-	T	2 ⁴⁰	Tebi-	Ti
1000 ⁵	Peta-	P	2 ⁵⁰	Pebi-	Pi
1000 ⁶	Exa-	E	2 ⁶⁰	Exbi-	Ei
1000 ⁷	Zetta-	Z	2 ⁷⁰	Zebi-	Zi
1000 ⁸	Yotta-	Y	2 ⁸⁰	Yobi-	Yi
1000 ⁹	Ronna-	R	2 ⁹⁰	Robi-	Ri
1000 ¹⁰	Quecca-	Q	2 ¹⁰⁰	Quebi-	Qi
1000 ⁻¹	milli-	m	1000 ⁻³	femto-	f
1000 ⁻²	micro-	μ	1000 ⁻⁶	atto-	a
1000 ⁻³	nano-	n	1000 ⁻⁹	zepto-	z
1000 ⁻⁴	pico-	p	1000 ⁻¹²	yocto-	y
			1000 ⁻¹⁵	ronto-	r
			1000 ⁻¹⁸	quecto-	q

RISC-V Reference Data Card ("Green Card") 1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together